# Java 0

**Programación Java en Linux (WSL) con terminal en VS Code**

## Qué vamos a construir

Este tutorial sigue el orden de la clase. Empezamos con dos archivos en `java-pr0` y los convertimos, paso a paso, en un proyecto con paquete, compilado en `bin` y comprimido en un zip. `Principal` usa los métodos `print` y `getInt` de `Auxiliar`.

Esta es la estructura final del proyecto:

```
java-pr0/
├── src/
│   └── dam/
│       ├── Auxiliar.java
│       └── Principal.java
├── bin/
│   └── dam/
│       ├── Auxiliar.class
│       └── Principal.class
└── jpiay-pr0.zip
```

Todos los comandos se lanzan desde la carpeta raíz, `java-pr0`. Si los lanzas desde otra carpeta, las rutas no coincidirán.

## 1. Punto de partida: dos clases en java-pr0

Empezamos con dos archivos sueltos en `java-pr0`, sin paquete. Para usar un método de otra clase basta con escribir `Clase.metodo()`.

```
mkdir -p ~/dam2627/prog/java-pr0
cd ~/dam2627/prog/java-pr0
touch Auxiliar.java Principal.java
code .
```

`Auxiliar.java` guarda los métodos reutilizables:

```
import java.util.Scanner;

public class Auxiliar {

    public static void print(String texto) {
        System.out.println(texto);
    }

    public static int getInt() {
        Scanner teclado = new Scanner(System.in);
        return Integer.parseInt(teclado.nextLine());
    }

}
```

- `public` permite usar el método desde otras clases.
- `static` permite llamarlo sin crear un objeto con `new Auxiliar()`. También es lo que hará posible el `import static` más adelante.
- `getInt()` lee una línea del teclado con `Scanner` (`teclado.nextLine()`) y la convierte en número entero con `Integer.parseInt`.

`Principal.java` tiene el `main` y llama a los métodos con el nombre de la clase delante:

```
public class Principal {
    public static void main(String[] args) {
        Auxiliar.print("Edad?");
        Auxiliar.print("Tienes " + Auxiliar.getInt() + " años");
    }
}
```

Así funciona. Se compila y ejecuta desde `java-pr0`, y los `.class` quedan junto a los `.java`:

```
javac *.java
java Principal
```

## 2. Quitar el nombre de la clase con import static

Escribir `Auxiliar.` delante de cada método cansa, así que probamos a importar sus métodos estáticos:

```
import static Auxiliar.*;

public class Principal {
    public static void main(String[] args) {
        print("Edad?");
        print("Tienes " + getInt() + " años");
    }
}
```

Al compilar, `javac` no encuentra la clase:

```
Principal.java:1: error: cannot find symbol
import static Auxiliar.*;
              ^
  symbol: class Auxiliar
```

El motivo es que Java no permite importar desde una clase sin paquete (el llamado paquete sin nombre). Un `import` siempre necesita la forma `paquete.Clase`, así que primero hay que poner las clases en un paquete.

## 3. Meter las dos clases en el paquete dam

Para que ambas clases pertenezcan al paquete `dam`, añade esta línea al principio de los dos archivos:

```
package dam;
```

- Debe ser la primera instrucción del archivo, antes de cualquier `import`.
- Las dos clases llevan la misma línea, así que comparten paquete.
- Desde ahora el nombre completo de las clases es `dam.Principal` y `dam.Auxiliar`.

## 4. Cada paquete en una carpeta con su nombre

Las clases del paquete `dam` deben estar en una carpeta llamada `dam`, porque Java busca `dam.Principal` en `dam/Principal.class`. Si dejas los archivos en la raíz, `java` no encuentra la clase:

```
Error: Could not find or load main class dam.Principal
Caused by: java.lang.ClassNotFoundException: dam.Principal
```

Desde `java-pr0`, crea la carpeta del paquete dentro de `src`, mueve allí los dos `.java` y borra los `.class` antiguos:

```
mkdir -p src/dam
mv Auxiliar.java Principal.java src/dam/
rm -f *.class
```

El proyecto queda así:

```
java-pr0/
└── src/
    └── dam/
        ├── Auxiliar.java
        └── Principal.java
```

## 5. Importar con el paquete: import static dam.Auxiliar.*

Con las clases ya en el paquete, el import lleva delante el nombre del paquete. Así queda `Principal.java`:

```
package dam;

import static dam.Auxiliar.*;

public class Principal {
    public static void main(String[] args) {
        print("Edad?");
        print("Tienes " + getInt() + " años");
    }
}
```

| Parte | Significado |
|---|---|
| `import static` | Importa miembros estáticos de una clase, no la clase |
| `dam` | Paquete donde está la clase |
| `Auxiliar` | Clase que contiene los métodos |
| `.*` | Todos sus miembros estáticos accesibles, aquí `print` y `getInt` |

`Auxiliar.java` solo cambia en su primera línea, `package dam;`.

El `main` se ejecuta en este orden:

1. `print("Edad?")` muestra la pregunta.
2. `getInt()` espera a que escribas la edad y devuelve el número.
3. Java une el texto y el número: `"Tienes 20 años"`.
4. `print(...)` muestra el resultado.

## 6. Compilar desde la raíz y dejar el bytecode en bin

Este comando traduce los `.java` a bytecode (archivos `.class`, lo que ejecuta la máquina virtual de Java) y los guarda en `bin`:

```
javac -d bin src/dam/*.java
```

| Parte | Significado |
|---|---|
| `javac` | Compilador de Java |
| `-d bin` | Carpeta de destino de los `.class`; si no existe, la crea |
| `src/dam/*.java` | Compila todos los `.java` de esa carpeta |

Si todo va bien, `javac` no muestra nada. `-d` crea la subcarpeta `dam` a partir de la línea `package dam;`, así que `bin` queda con la misma estructura que el paquete. Compruébalo con:

```
ls -R bin
```

```
bin:
dam

bin/dam:
Auxiliar.class  Principal.class
```

Sin `-d bin`, los `.class` se quedarían mezclados con los `.java` en `src/dam`. Vuelve a compilar cada vez que cambies un `.java`: `java` ejecuta los `.class`, no tu código fuente.

## 7. Ejecutar desde el terminal

Para ejecutar el programa, indica dónde está el bytecode y el nombre completo de la clase que tiene el `main`:

```
java -cp bin dam.Principal
```

| Parte | Significado |
|---|---|
| `java` | Arranca la máquina virtual |
| `-cp bin` | Classpath: la carpeta donde buscar las clases |
| `dam.Principal` | Nombre completo de la clase: paquete, punto y clase |

Java busca `dam.Principal` en `bin/dam/Principal.class`. Por eso se escribe con punto, sin `.class` y sin la ruta. Si escribes 20 y pulsas Intro, verás:

```
Edad?
20
Tienes 20 años
```

## 8. Comprimir el proyecto

Este comando crea `jpiay-pr0.zip` con las carpetas `src` y `bin` y todo lo que contienen:

```
zip -r jpiay-pr0.zip src/ bin/
```

| Parte | Significado |
|---|---|
| `zip` | Programa de compresión |
| `-r` | Recursivo: incluye subcarpetas y archivos; sin él, solo guarda las carpetas vacías |
| `jpiay-pr0.zip` | Nombre del archivo que se crea |
| `src/ bin/` | Lo que se comprime |

Compila antes de comprimir para que el zip lleve el bytecode actualizado. `deflated 43%` en la salida significa que ese archivo ocupa un 43 % menos dentro del zip.

Si el zip ya existe, el comando lo actualiza en lugar de crearlo de cero, y lo que hayas borrado del proyecto sigue dentro. Para empezar limpio, bórralo antes con `rm jpiay-pr0.zip`.

## 9. Ver lo comprimido

`unzip -l` lista el contenido del zip sin extraer nada:

```
unzip -l jpiay-pr0.zip
```

Esta es la salida con el zip de clase:

```
Archive:  jpiay-pr0.zip
  Length      Date    Time    Name
---------  ---------- -----   ----
        0  2026-11-06 12:32   src/
        0  2026-11-06 12:33   src/dam/
      194  2026-11-06 17:30   src/dam/Principal.java
      306  2026-11-06 11:32   src/dam/Auxiliar.java
        0  2026-11-06 12:40   bin/
        0  2026-11-06 12:40   bin/dam/
      830  2026-11-06 18:02   bin/dam/Principal.class
      670  2026-11-06 18:02   bin/dam/Auxiliar.class
---------                     -------
     2000                     8 files
```

- **Length** es el tamaño original en bytes; las carpetas aparecen con 0.
- **Date** y **Time** son la última modificación de cada elemento.
- **Name** es la ruta dentro del zip.
- **2000** es la suma de los tamaños: 194 + 306 + 830 + 670.
- **8 files** cuenta todas las entradas: 4 archivos y 4 carpetas.

El zip está completo: lleva las dos clases en fuente y en bytecode. Para comprobar que también funciona, extráelo en otra carpeta y ejecútalo desde allí:

```
unzip jpiay-pr0.zip -d prueba
java -cp prueba/bin dam.Principal
rm -r prueba
```

## Errores frecuentes

| Lo que escribes o ves | Qué pasa | Solución |
|---|---|---|
| `java -cp bin Principal` da `ClassNotFoundException: Principal` | Falta el paquete en el nombre | `java -cp bin dam.Principal` |
| `java bin/dam/Principal` da `wrong name: dam/Principal` | Has puesto una ruta en lugar del nombre de la clase | `java -cp bin dam.Principal` |
| `java Principal` dentro de `bin/dam` da `wrong name: dam/Principal` | Estás en la carpeta equivocada | Vuelve a la raíz con `cd ../..` |
| `javac src/dam/Principal.java` da `cannot find symbol` | Compilas una sola clase y no encuentra `Auxiliar` | `javac -d bin src/dam/*.java` |
| Sale `cannot find symbol` en `print` o `getInt` | Falta `import static dam.Auxiliar.*;` | Añádelo después de `package dam;` |
| Al escribir la edad sale `NumberFormatException: For input string: "veinte"` | `Integer.parseInt` solo convierte textos que son números | Escribe la edad en cifras: `20` |
| Sale `Tienes 20 a?os` | El terminal no usa UTF-8 | Ejecuta `export LC_ALL=C.UTF-8` y repite |

## Resumen de comandos

Desde la carpeta `java-pr0`:

```
# Compilar
javac -d bin src/dam/*.java

# Ejecutar
java -cp bin dam.Principal

# Comprimir
zip -r jpiay-pr0.zip src/ bin/

# Ver lo comprimido
unzip -l jpiay-pr0.zip
```